

# i2i Academy

## Training Document

|               |                  |
|---------------|------------------|
| Topic         | VoltWise         |
| Document Name | VoltWise Project |

## Document History

| Date       | Author      | Ver | Comments          |
|------------|-------------|-----|-------------------|
| 04.06.2026 | i2i Academy | 1.0 | Initial Creation. |

## ***Document History***

### **1. Difficulty Level**

### **2. General Requirements**

- 2.1. Code Quality
- 2.2. Documentation and Homework Submission
- 2.3. Code Management and Portfolio Building (GitHub)
- 2.4. Submission Process (Google Classroom)
- 2.5. Tracking and Reporting (Google Sheets)
- 2.6. Academic Integrity

### **3. Description**

### **4. Objective**

### **5. Requirements**

- 5.1. VoltWise Web App
  - 5.1.1 Functional Requirements
  - 5.1.2 Non-Functional Requirements
- 5.2 VoltWise Core
  - 5.2.1 Functional Requirements
  - 5.2.2 Non-Functional Requirements
- 5.3 VoltWise Telemetry Sensors
  - 5.3.1. Telemetry Data Generation
  - 5.3.2. Asset Registration Consumption

### **6. Deliverables**

## 1. Difficulty Level

| Beginner | Junior | Mid | Senior | Expert |
|----------|--------|-----|--------|--------|
| •        | •      | •   | •      | •      |

## 2. General Requirements

Compliance with the following standards is mandatory for all homework assignments. These rules aim to ensure a fair evaluation process and instill professional working discipline.

### 2.1. Code Quality

Code must be readable, modular, and compliant with Clean Code principles. Avoid redundant comments and use meaningful naming conventions for variables and functions.

### 2.2. Documentation and Homework Submission

Every assignment must be documented with clear explanations and screenshots. Upon completion of the tasks, the designated Solution section (Section 4) must be filled out completely. The final step is to upload and submit this completed document exclusively via Google Classroom.

### 2.3. Code Management and Portfolio Building (GitHub)

Every homework assignment must have a "Public" repository on GitHub. Repositories must be named following the i2i-Academy-[HomeworkName]-[HomeworkNo] (e.g., i2i-Academy-ConnectingACloudService-1) format. Code must be uploaded at regular intervals with meaningful commit messages; avoid "bulk uploads" at the end of the project.

- **Important Note:** The primary goal of using GitHub is to help you build a professional portfolio for your future career. Therefore, **do not upload this homework document** (instructions, screenshots, etc.) to your GitHub repository. You may explain the purpose and scope of the project in your README.md file, but the repository must contain only your actual source code and configuration files.

### 2.4. Submission Process (Google Classroom)

Official notification of homework completion must be made exclusively via Google Classroom. The GitHub repository link must be provided in the assignment submission, and the process must be finalized by clicking the "Turn In" button. Submissions made after the specified deadline may not be evaluated unless prior approval is obtained.

### 2.5. Tracking and Reporting (Google Sheets)

For each completed task, the checkbox of the corresponding homework must be checked in the shared Google Sheet. Ensuring consistency between the Classroom submission and the Google Sheet update is the student's responsibility.

### 2.6. Academic Integrity

All submitted work must be original. Copying code from other students or uncredited sources is strictly prohibited.

### 3. Description

**VoltWise** is a real-time IoT energy analytics and budget auditing platform designed to monitor household electricity consumption. The system processes continuous data streams from smart home appliances, providing users with immediate visibility into their overall power metrics and ongoing utility costs.

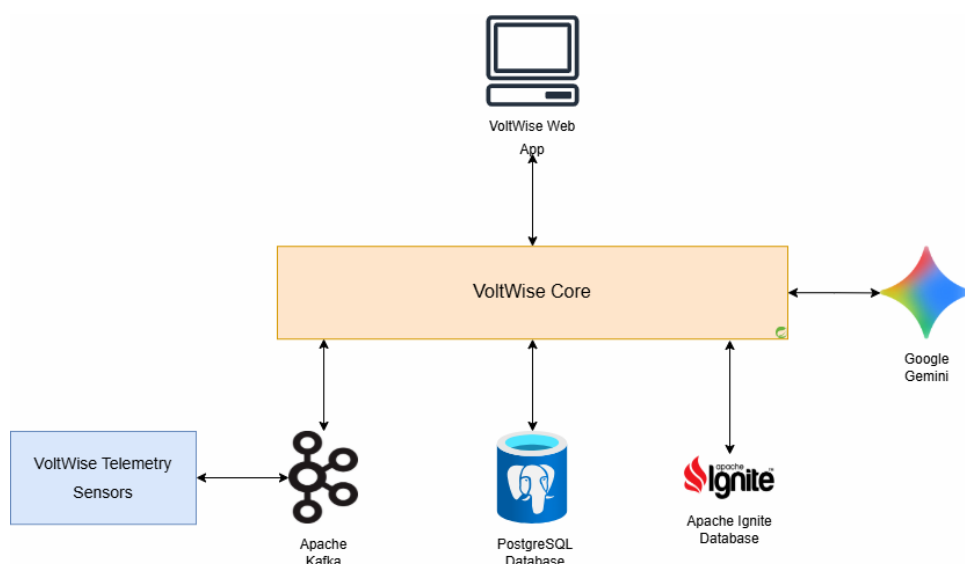
The platform fundamentally operates as an automated tracking and early-warning ecosystem. It continuously evaluates accumulated energy usage and financial spend against predefined target quotas, while simultaneously tracking individual appliances using high-speed in-memory consecutive breach counters to catch immediate power limit violations. Upon a home breaching its 100% budget threshold, the platform dynamically enforces a premium penalty tariff tier, instantly accelerating subsequent billing rates. When these critical quota breaches or recurring device anomalies are detected, **VoltWise** orchestrates an automated notification pipeline that utilizes generative artificial intelligence to synthesize and dispatch personalized, tailored email alerts, guiding consumers to optimize their usage habits and prevent financial overruns.

## 4. Objective

The primary technical objective is to engineer a high-performance, low-latency, and high-throughput modular backend architecture that successfully delivers **VoltWise**'s functional capabilities.

This will be achieved by implementing a decoupled, event-driven ecosystem utilizing the following software engineering components and strategic designs:

- **VoltWise Web App** - The client-side frontend interface that interacts with the VoltWise Core to deliver a responsive monitoring dashboard, entirely decoupled from the backend technology stack to maintain a clean separation of concerns.
- **VoltWise Core** - The central Spring Boot monolith responsible for orchestrating real-time streaming data, evaluating live threshold metrics, enforcing dynamic billing rules, and driving the LLM prompt pipeline.
- **VoltWise Telemetry Sensors** - An autonomous background simulation engine that mocks physical IoT appliances, eliminating direct core dependencies by operating strictly through an asynchronous event loop.
- **Apache Kafka** - A distributed message broker managing a bidirectional event architecture via separate registration and telemetry topologies to guarantee scalable, loose coupling.
- **Apache Ignite** - An In-Memory Data Grid (IMDG) serving as the live state-tracking and execution tier. It manages active appliance counters, handles instant penalty computations, and serves high-frequency UI polling requests with sub-millisecond latency to shield the persistent database.
- **PostgreSQL Database** - A persistent relational database optimized to maintain structured data requiring strict ACID compliance, including master registries, historical logs, financial audit trails, and permanent LLM recommendations.
- **Google Gemini** - An LLM integration engine utilized for dynamic prompt composition, transforming technical grid metrics into personalized, behavioral email alerts.



## 5. Requirements

### 5.1. VoltWise Web App

The **VoltWise Web App** serves as the client-side Single-Page Application (SPA) user interface of the ecosystem. Its primary role is to provide a clean, responsive, and intuitive interface for users to monitor ongoing electricity consumption profiles, observe escalating utility costs, and track AI-generated behavioral alert streams.

### 5.1.1 Functional Requirements

- **Real Time Dashboard Grid** - The interface must display a comprehensive grid or list of all registered residential structures. Clicking on a specific home card must trigger an interactive modal popup that dynamically renders the real-time consumption metrics of all associated appliances along with the total cumulative billing information.
- **Dynamic Quota Breach and Anomaly Identification** - The application must visually differentiate homes that have breached their predefined power or financial budget quotas on the main dashboard layout. Within the detailed home modal view, specific appliances marked with anomalous metrics or consecutive threshold violations must be rendered with distinct visual treatments to instantly separate them from normally functioning devices.
- **Interactive Analytical Charts** - The interface must dynamically render periodic charts for individual residential structures using a visualization library. These charts must be embedded directly inside the detailed home modal view, positioned underneath the live appliance consumption list, to illustrate daily historical energy consumption trends and consumption fluctuations per home based on incoming telemetry packets.

### 5.1.2 Non-Functional Requirements

- **UI Fluidity During Aggressive Polling** - The application interface must remain fully interactive and fluid during active 1 to 2 second polling intervals. High frequency updates to the DOM and chart components must not cause user interface freezing, layout shifts, or input responsiveness lag.
- **Asynchronous Loading Indication** - The web application must explicitly display visual state indicators like skeleton components or processing spinners during network bound executions. These indicators are required for live analytical data streams to load.
- **Graceful Client Side Error Interception** - The frontend must intercept and gracefully process human readable error responses triggered by network connectivity drops or backend input validation failures. Raw backend system exceptions and stack traces must be completely suppressed and replaced with user friendly alerts or toast warnings.

## 5.2 VoltWise Core

The **VoltWise Core** acts as the central backend application of the ecosystem. It serves as a modular monolith responsible for orchestrating modules, processing real-time telemetry streams, managing billing rules, and formatting dynamic contextual prompts for the LLM layer. Modularizing the core ensures clear boundaries, making it easier to understand how the backend serves each specific architectural component.

### 5.2.1 Functional Requirements

- **Home and Metrics Management Module** - This module handles residential data structures and acts as the bridge to both **Apache Ignite** and **PostgreSQL** for live monitoring:
  - **Home Registration Endpoint** - Expose a public REST POST endpoint to register new residential structures along with their appliance topologies via **Swagger**. This endpoint must persist the structure to **PostgreSQL** and immediately publish the asset configuration event to the **Apache Kafka** registration topic.
  - **Home Status Delivery** - Expose a public REST endpoint for the frontend to poll the latest home metrics. This endpoint must pull data exclusively from **Apache Ignite** to guarantee real-time accuracy and eliminate database read overhead.
  - **Historical Trend Delivery** - Expose a public REST endpoint to serve daily consumption history for the frontend charts. This endpoint must fetch aggregated historical snapshots exclusively from **PostgreSQL**.
- **Telemetry Processing Module** - This module manages high-frequency IoT data stream ingestion and in-memory state tracking:
  - **Asynchronous Stream Processing** - Continuously ingest JSON telemetry payloads from the **Apache Kafka** topic and extract active appliance consumption metrics.
  - **In-Memory State Updates** - Atomically update the **Apache Ignite** with the accumulated energy consumption and billing metrics for each home upon receiving every telemetry message.
- **Tariff and Anomaly Rules Module** - This module enforces the core business logic, billing rules, and real-time device checks:
  - **Quota Evaluation** - Evaluate live billing totals against user quotas. Trigger the alert pipeline immediately when a home reaches 80% or 100% of its limits.
  - **Dynamic Penalty Tariff** - Automatically transition a home into a penalty state in **Apache Ignite** once it breaches the 100% budget quota, calculating all subsequent usage at a premium penalty rate.
  - **Consecutive Breach Counter** - Maintain individual device violation counters in **Apache Ignite**. If an appliance exceeds its safe limit for 3 consecutive cycles, mark it as anomalous and trigger an alert. Reset the counter when it returns to normal.
- **AI Notification Module** - This module handles prompt composition and communication with the artificial intelligence layer:
  - **Prompt Orchestration** - Gather the home metrics, billing totals, and active anomaly data from **Apache Ignite** and **PostgreSQL**. Assemble this context into a structured prompt layout and transmit it to LLM to synthesize a personalized behavioral advisory text in Turkish.
  - **Email Dispatch & Logging** - Persist the AI-generated recommendation into **PostgreSQL** and automatically dispatch the tailored email notification to the contact email address defined for the home.

### 5.2.2 Non-Functional Requirements

- **Single-Application Modularity** - The core must be developed entirely as a single Spring Boot application. Modularity must be achieved via a clean package structure based on functionality.
- **Operational Sequencing and Integrity** - All critical status updates and logging operations must enforce robust error boundaries. The system must update the active in-memory state inside **Apache Ignite** before initiating historical event logging in **PostgreSQL Database**, utilizing exception handling to log failures and prevent data fragmentation.
- **Environment Configuration & Security** - No sensitive keys, database credentials, or the **LLM** API key may be hardcoded. The application must extract all configuration parameters from environment variables.
- **Resilient Error Fallbacks** - The AI routing logic must implement try-catch fallbacks. If the external **LLM** API is unreachable or rate-limited, the system must return a clean fallback instead of hanging or blocking the main application server and **Kafka** consumer threads.

## 5.3 VoltWise Telemetry Sensors

This component acts as the autonomous background simulation engine responsible for mocking physical smart home appliances. It must be completely decoupled from the core business logic, communicating strictly through an asynchronous event infrastructure.

### 5.3.1. Telemetry Data Generation

- Run an isolated background task that automatically generates realistic power consumption fluctuations in Watt for registered appliances every few seconds. Serialize these live metrics into JSON format and publish them directly to the **Apache Kafka** telemetry topic.

### 5.3.2. Asset Registration Consumption

- Maintain an active event listener that automatically reads new home provisioning records from the **Apache Kafka** registration topic. Dynamically append these newly discovered houses and their appliance definitions to the running simulation memory.

## 5.4 Apache Ignite

**Apache Ignite** serves as the high-speed In-Memory Data Grid (IMDG) for volatile and high-frequency operational metrics. It acts as the performance and state execution layer to handle rapid telemetry updates and offload read operations from the **PostgreSQL** database.

### 5.4.1. Functional Requirements

- **Live State Tracking** - Store active home telemetry profiles containing cumulative energy usage, rolling billing totals, and active tariff states to allow instant lookups for subsequent interface requests.

- **Consecutive Breach Monitoring** - Maintain real-time consecutive power limit violation counters mapped to specific appliance keys, ensuring anomaly thresholds are evaluated completely in-memory with zero disk overhead.
- **Atomic State Overwriting** - Accept high-frequency updates from the VoltWise Core module to incrementally add consumption costs, transition homes to penalty tariff rates, or reset device counters immediately upon processing telemetry events.

#### 5.4.2. Non-Functional Requirements

- **Sub-Millisecond Latency** - Serve all dashboard endpoint read requests and state checks with sub-millisecond response times to prevent high-frequency user interface polling from bottlenecking the backend system.
- **Volatile State Isolation** - Restrict in-memory data grid structures to ephemeral consumption states and active operational flags. Because **PostgreSQL** remains the single source of truth for permanent financial records, **Apache Ignite** must never store permanent audit trails or historical daily logs.

### 5.5 Apache Kafka

**Apache Kafka** serves as the distributed message streaming backbone for the ecosystem. It enables loose coupling and asynchronous communication between the backend application and the telemetry simulation engine.

#### 5.5.1. Functional Requirements

- **Telemetry Stream Topic** - Handle high-frequency event streams where the **VoltWise Telemetry Sensors** publish live appliance wattage metrics and the **VoltWise Core** consumes them for billing and anomaly processing.
- **Asset Registration Topic** - Manage lifecycle events where the **VoltWise Core** publishes newly created home configurations and the **VoltWise Telemetry Sensors** consume them to dynamically add houses to the running simulation memory.

#### 5.5.2. Non-Functional Requirements

- **Asynchronous Decoupling** - Ensure complete separation between modules so that a temporary downtime or performance lag in the **VoltWise Core** does not interrupt or block the active event loop of the **VoltWise Telemetry Sensors**.

### 5.6 PostgreSQL Database

**PostgreSQL Database** serves as the persistent relational database of the ecosystem. It acts as the immutable single source of truth for all permanent, structured data that requires strict transactional consistency.

### 5.6.1 Functional Requirements

- **Home and Appliance Persistence** - Permanently store structured data for registered residential structures, including a target contact email address field, and their static appliance configurations created during system initialization.
- **Billing and Quota Archiving** - Maintain accurate permanent ledger records of baseline home budget constraints, current accumulated monetary balances, and utility rates.
- **Transactional Event Logging** - Record an unalterable history of all critical operational events, including exact timestamps of 80% and 100% quota breaches, dynamic penalty tariff activations, and generated AI advisory metadata.
- **Consumption Trend Logging** - Periodically record and store historical energy consumption snapshots pushed by the backend core to provide a persistent source for frontend daily trend chart analysis.

### 5.6.2 Non-Functional Requirements

- **Transactional Integrity (ACID)**: Ensure all historical ledger entries and event logs strictly adhere to ACID properties. If a database operation fails within a multi-step PostgreSQL insert, the local database transaction must roll back completely to ensure relational data consistency.
- **Relational Constraints** - Implement proper database schemas with strict foreign key constraints to ensure that appliance records, billing logs, and AI notifications are always tied to a valid, existing residential profile.
- **Data Reliability** - Serve as the absolute authority for financial and consumption state. Unlike the ephemeral **Apache Ignite** grid, all data written to **PostgreSQL Database** must remain fully intact across application restarts or server resets.

## 5.7 LLM

Google Gemini serves as the artificial intelligence layer of the ecosystem. It is responsible for transforming raw energy metrics and anomaly states into personalized behavioral saving recommendations for users.

### 5.7.1. Functional Requirements

- **Contextual Prompt Composition** - Accept compiled data payloads containing home-specific billing totals, quota breach states, and appliance anomaly metrics fetched from **Apache Ignite** and **PostgreSQL**.
- **Automated Alert Generation** - Process the composed prompt to generate actionable, personalized energy-saving advice written strictly in the Turkish language.

- **Structured Response Delivery** - Output all generated recommendations in a clean text format that can be seamlessly processed and included in automated email notifications.

### 5.7.2. Non-Functional Requirements

- **Resilient Error Fallbacks** - Implement strict exception handling to capture API timeouts, rate limits, or connection failures. If the LLM is unavailable, the system must return a clean fallback text instead of hanging the application server or consumer threads.
- **Secure Token Management** - The external Gemini API key must be injected at runtime exclusively via environment variables and must never be hardcoded or exposed to the client-side application.

## 6. Deliverables

The following concrete deliverables are expected to be completed and submitted via a dedicated GitHub repository within a strict 1-week timeline:

- **VoltWise Web App Source Code** - The complete source code for the Single-Page Application (SPA) monitoring dashboard.
- **VoltWise Core Source Code** - A single Spring Boot application utilizing a clean, modular package structure with well-decoupled layers, fully configured via environment variables.
- **Database Schema and Scripts** - Database definition scripts (DDL) required to generate the necessary **PostgreSQL** tables with proper constraints, integrated into the Docker setup to execute automatically on container setup.
- **Infrastructure Configuration** - A single, ready-to-run docker-compose.yml file to effortlessly provision and spin up local **PostgreSQL**, **Apache Kafka**, and **Apache Ignite** container instances.
- **API Documentation** - A comprehensive **Swagger** or **OpenAPI** documentation to allow immediate testing of all exposed backend endpoints.
- **Project README** - A precise README.md file detailing step-by-step instructions on local repository setup, environment variable configuration, and application launch commands.